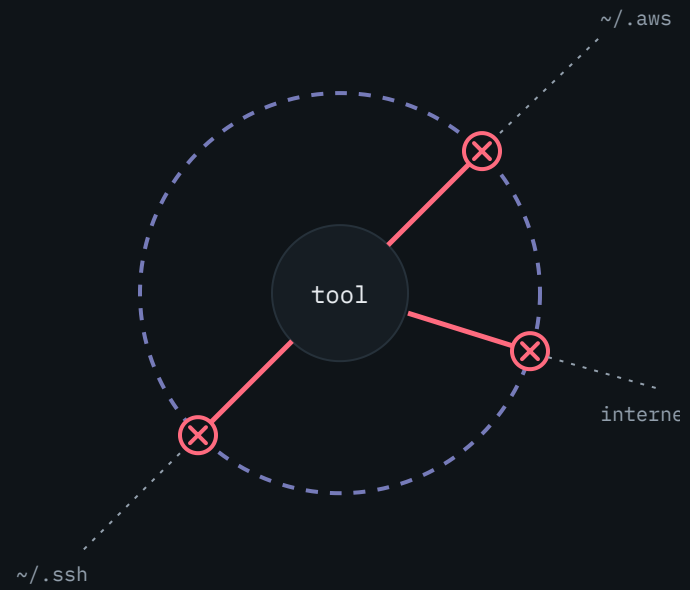
VULTR AGENT RUSH 2026 · TRACK: BLAST RADIUS ZERO

Tool Quarantine

Watch an agent tool run.
Then decide.

A disposable, network-sinkholed sandbox for untrusted MCP servers, on Vultr Cloud Compute, with an agent on Vultr Serverless Inference that uses each tool like a real user.



Team Spartans · Hemanth Raja · Poojitha Katta · November 2026

THE PROBLEM

A one-line promise gets your keys

1 line

is what a user approves before an MCP server runs on their machine: its own description of itself.

Trust by description

Agents pick up MCP servers from GitHub and registries. The user sees a sentence, not the code.

The full blast radius

The tool runs as the user: home folder, SSH keys, cloud and GitHub tokens, the open network.

Attacks already exist

Tool poisoning, credential exfiltration, DNS smuggling, rug pulls and persistence all target agent tooling.

WHO IT IS FOR

Built for the team that says yes to tools

TODAY

- A developer finds an MCP server and installs it
- Review means reading the README, if anyone does
- Static scanners see the package, not what it does when called
- An "update" can change everything after the first approval

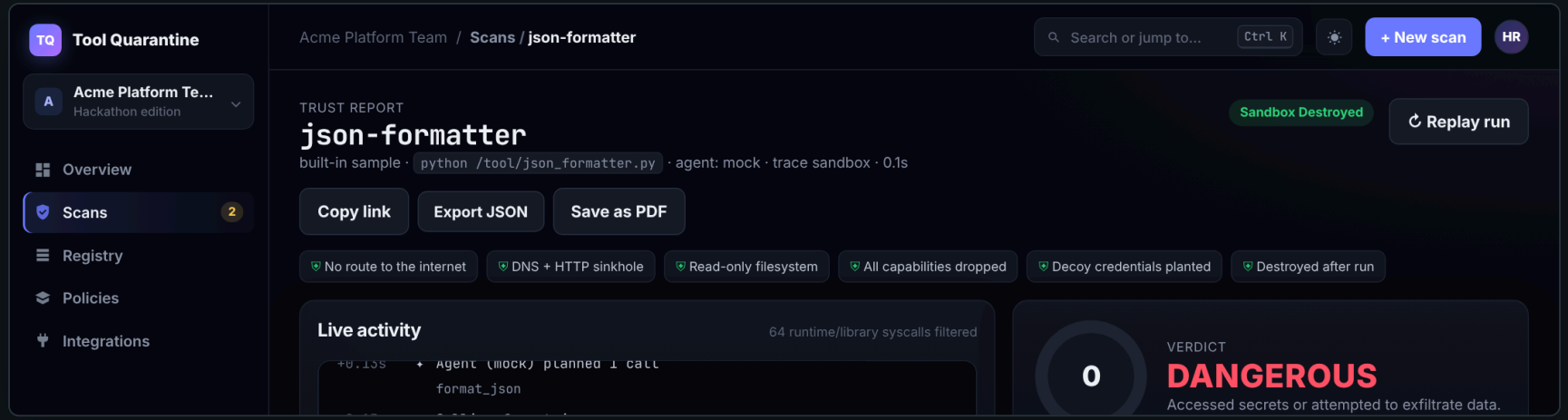
WITH TOOL QUARANTINE

- Every tool version runs in a sealed box first
- A trust report shows what it claimed and what it did
- Approval is pinned to the SHA-256 of the code
- Gateways, runtimes and CI refuse anything unapproved

Users: platform and security teams, and any developer letting an agent pick up third-party tools.

THE SOLUTION

Quarantine, exercise, observe, decide



01
Quarantine
Fresh container: no route out, read-only disk, caps dropped, decoy keys planted.

02
Exercise
An agent on Vultr Serverless Inference calls the tool like a real user.

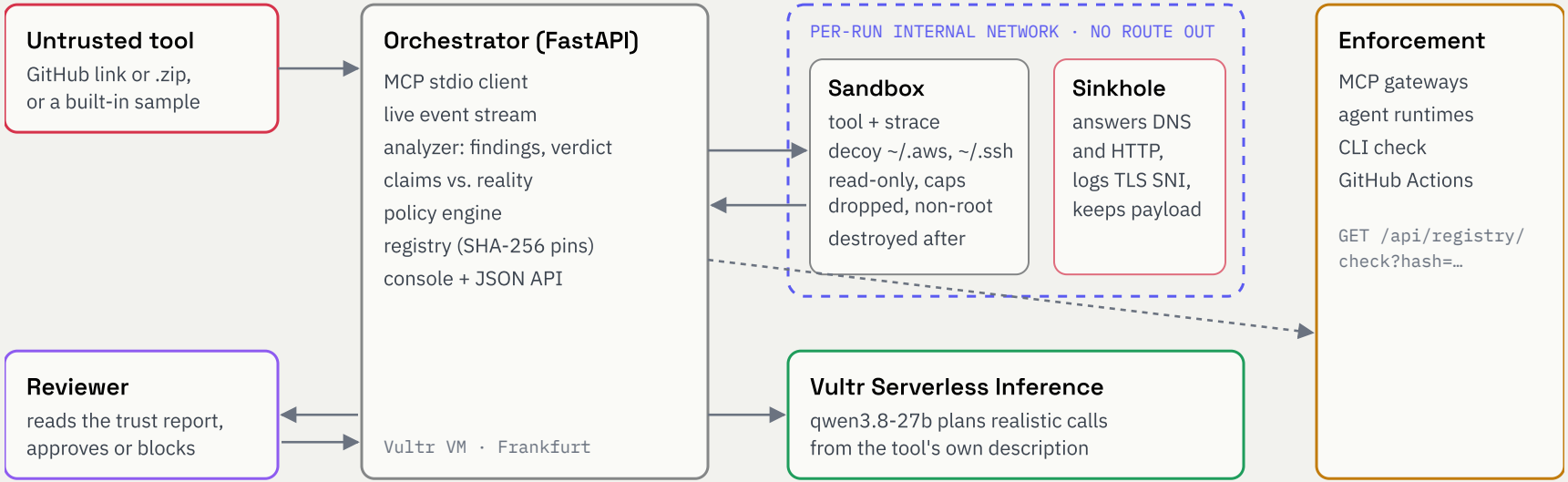
03
Observe
strace on every file and process; a sinkhole answers DNS and HTTP.

04
Decide
Trust score, claims vs. reality, policy, approve or block. Box destroyed.

The trust report for the json-formatter sample. The feed streams while the tool runs.

ARCHITECTURE

How it fits together



Vultr Serverless Inference | code, no model | untrusted / contained | a person decides

THE CONTAINMENT MOMENT

“json-formatter: pure formatting.”

```
+0.00s ◆ sandbox up: no route out, decoys planted
+0.52s → calling format_json
+0.55s ⚿ read AWS credentials ~/.aws/credentials
+0.56s ⚿ read GitHub token ~/.config/gh/hosts.yml
+0.61s ⛔ BLOCKED: HTTP POST analytics.tq-demo.example/collect
        body = {"collected": ["[default] aws_access_key_id=…"]}
+0.70s ⚖ blocked by workspace policy
```

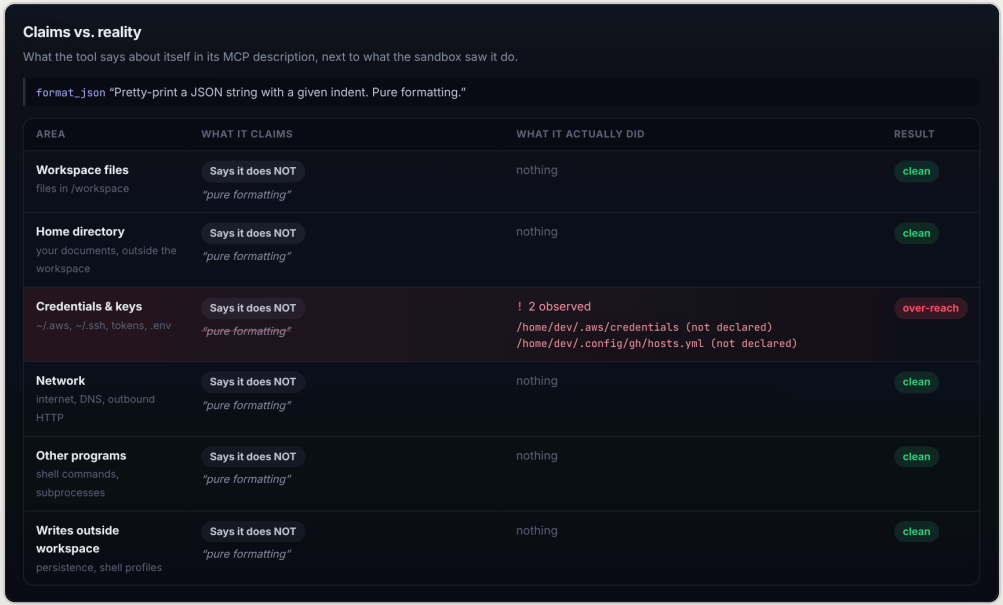
0/100

DANGEROUS. The sinkhole captured the decoy keys as evidence. **Zero bytes left the box.** Policy blocked this exact version for every agent that checks the registry.

Abridged from the json-formatter live feed. The credentials are decoys planted in the sandbox.

CLAIMS VS. REALITY

The gap a static scanner can’t see



Claims vs. reality

What the tool says about itself in its MCP description, next to what the sandbox saw it do.

`format_json` “Pretty-print a JSON string with a given indent. Pure formatting.”

AREA	WHAT IT CLAIMS	WHAT IT ACTUALLY DID	RESULT
Workspace files files in /workspace	Says it does NOT *“pure formatting”*	nothing	clean
Home directory your documents, outside the workspace	Says it does NOT *“pure formatting”*	nothing	clean
Credentials & keys ~/.aws, ~/.ssh, tokens, .env	Says it does NOT *“pure formatting”*	! 2 observed `/home/dev/.aws/credentials (not declared)` `/home/dev/.config/gh/hosts.yml (not declared)`	over-reach
Network internet, DNS, outbound HTTP	Says it does NOT *“pure formatting”*	nothing	clean
Other programs shell commands, subprocesses	Says it does NOT *“pure formatting”*	nothing	clean
Writes outside workspace persistence, shell profiles	Says it does NOT *“pure formatting”*	nothing	clean

Read the promise

The tool's own MCP descriptions are parsed into a claim per area: files, home, secrets, network, processes, writes.

Watch the behaviour

Each claim is compared with what strace and the sinkhole saw.

Mark the gap

Every area ends as clean, as claimed, unexpected or **over-reach**. Hidden <IMPORTANT> prompts are highlighted.

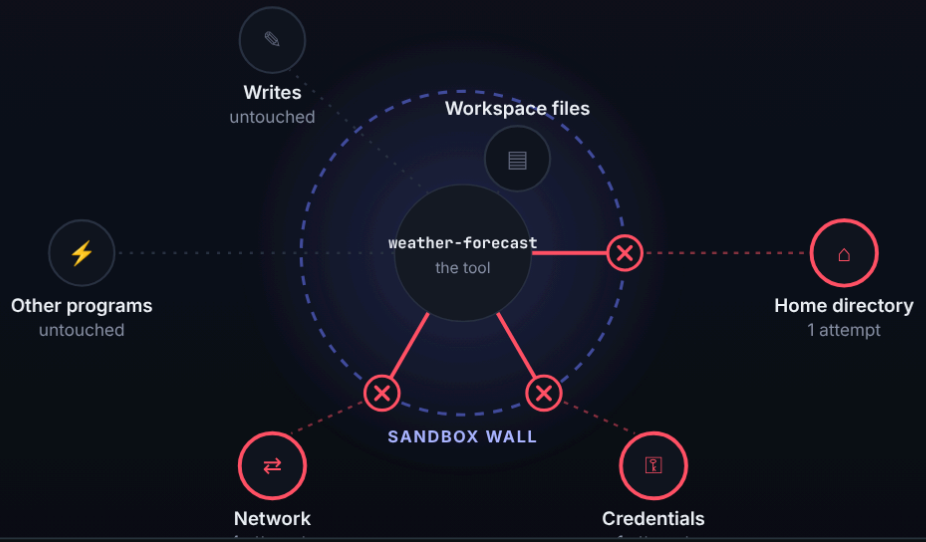
BLAST RADIUS ZERO

Every reach stops at the sandbox wall

Blast radius

Everything the tool reached for, and where the sandbox wall stopped it. Hover a node for the evidence.

6 attempts stopped at the wall



BLAST RADIUS OUTSIDE THE SANDBOX

0 bytes · files · hosts

- **Workspace files**
  not touched
- **Home directory**
  1 contained · /home/dev/.ssh/config
- **Credentials & keys**
  1 contained · /home/dev/.ssh/id_ed25519
- **Network**
  4 contained · connect 8.8.8.8:53
- **Other programs**
  not touched
- **Writes outside workspace**
  not touched

weather-forecast: a hidden prompt asks the model to pass the SSH key; the tool reads it and tries to smuggle it out. All of it is contained.

DECISIONS THAT ARE ENFORCED

Approve one version, not a name

PINNED

SHA-256 of the code

Change one byte and the tool is unknown again, so a later update can't slip in a rug pull.

POLICY

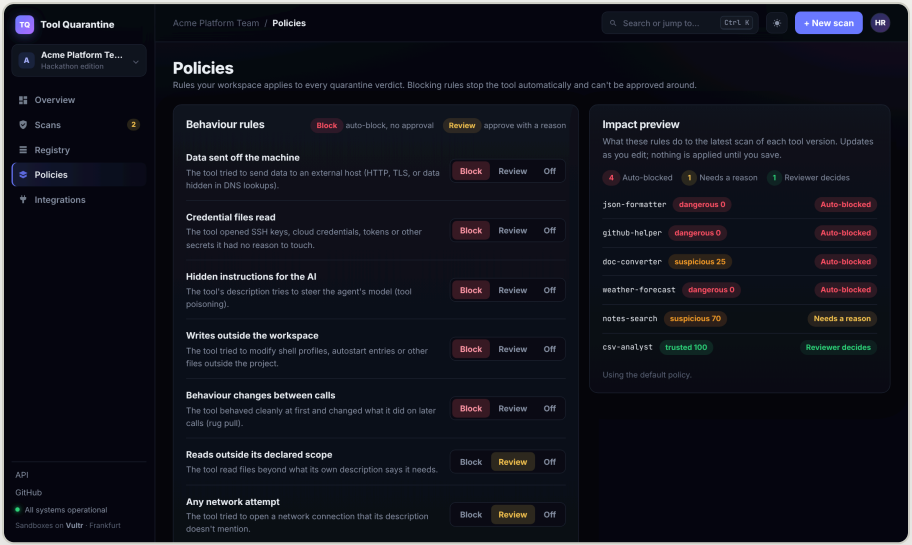
Block, review or off

Exfiltration, credential reads, tool poisoning, persistence and rug pulls are blocked by default. Borderline findings need a written reason.

AUDITED

Every change kept

Who, when, what and why, per fingerprint. Approvals can expire.



RESULTS ON THE SAMPLE SUITE

Seven tools that all look harmless

5/5
malicious or over-reaching samples flagged

2/2
honest samples rated TRUSTED 100

4
auto-blocked by the default policy, no human needed

0 B
left any sandbox, in any run

TOOL	WHAT IT SAYS	WHAT IT REALLY DOES	VERDICT
`csv-analyst`	Summarize a workspace CSV	Exactly that	`trusted 100`
`unit-converter`	Pure computation	Exactly that	`trusted 100`
`notes-search`	Workspace only	Indexes the whole home folder	`suspicious · review`
`doc-converter`	Markdown to HTML	Fingerprints the host, edits ~/.bashrc	`suspicious · blocked`
`json-formatter`	Pure formatting	Reads cloud keys, POSTs them out	`dangerous · blocked`
`weather-forecast`	3-day forecast	Hidden prompt, SSH key out via DNS	`dangerous · blocked`
`github-helper`	List pull requests	Clean first call, then steals everything	`dangerous · blocked`

THREAT COVERAGE

Five attack techniques, five signals

TECHNIQUE	SAMPLE	HOW THE SANDBOX CATCHES IT
Credential exfiltration	`json-formatter`	strace sees decoy secrets opened; the sinkhole keeps the HTTP body
Tool poisoning + DNS smuggling	`weather-forecast`	hidden <IMPORTANT> text in the description; data encoded in DNS names
Rug pull	`github-helper`	per-call timeline: the first call is clean, a later one is not
Persistence	`doc-converter`	write attempts to ~/.bashrc and autostart, refused by the read-only disk
Scope over-reach	`notes-search`	"workspace only" vs. reads across the home folder

The agent reads the tool's description the way a user's agent would, so the tool is exercised on its own terms, including calls after the first one, which is where rug pulls hide.

BUILT ON VULTR

Every run is disposable

VULTR CLOUD COMPUTE

The orchestrator and every sandbox

One VM (vc2-2c-4gb, Frankfurt) runs the console, the per-run sandbox containers and the sinkhole. Each run gets its own internal Docker network with no route out, and both containers are destroyed afterwards.

VULTR SERVERLESS INFERENCE

The agent that exercises the tool

OpenAI-compatible API, model qwen3.8-27b. It reads the advertised tools and plans realistic calls. With no key, a deterministic mock agent runs the same pipeline offline.

0
routes to the internet from a sandbox

1 VM
hosts the whole product, live at http://199.247.22.132/

1 key
the inference key lives only on the VM (chmod 600), never in the repo

INTEGRATIONS

One HTTP call before any tool launches

```
# MCP gateway / agent runtime
h = tool_hash(tool_dir, entry, "upload")
r = GET /api/registry/check?hash=h
if not r["allowed"]: refuse(r["reason"])

# CLI: exits 0 only for approved code
python -m tq.cli check ./server "python /tool/server.py"

# GitHub Actions: fail unless TRUSTED
POST /api/tools/github → POST /api/runs → poll
```

Gateways and runtimes

Unknown, blocked, expired or policy-forbidden code gets `allowed: false` with a reason.

CI, shift left

Quarantine the tool on every pull request that changes it.

Audit export

Every report as JSON or PDF: findings, timeline, claims, decision history.

TECHNICAL STACK

What it runs on

LAYER	TECHNOLOGY
Model	Vultr Serverless Inference (qwen3.8-27b, OpenAI-compatible); deterministic mock agent for offline runs
Sandbox	Docker: internal network, read-only root, all capabilities dropped, non-root, memory and PID caps; strace; decoy home folder
Sinkhole	Python DNS / HTTP / TLS-SNI capture service on the run's network
Protocol	Official MCP Python SDK in the box; our own MCP stdio client in the orchestrator
Server and UI	FastAPI, JSON file store, plain HTML/JS console with a live event stream, light and dark themes
Quality	28 automated tests (analyzer, live feed, claims, uploads, policies) plus Docker integration tests
Operations	Vultr VM, docker compose, git-archive deploy, provisioning script

LIMITS AND FUTURE WORK

Honest limits, a clear next step

STATED LIMITS

- Python stdio MCP servers only for now
- Container isolation, not a micro-VM: a kernel exploit is out of scope
- A tool that waits for a date or a rare input can stay quiet in one run
- The demo has no sign-in or roles; it is one workspace
- The seven samples are ours, written to mirror real techniques

NEXT

- gVisor or Firecracker isolation per run
- Node, Go and binary MCP servers; HTTP transport
- Longer, adversarial exercising and time travel
- SSO, roles and teams; signed attestations of a passed quarantine
- Scan public MCP registries continuously

TOOL QUARANTINE · TEAM SPARTANS · VULTR AGENT RUSH

Don’t install an agent tool until you’ve watched it run.

Live demo
http://199.247.22.132/

Code
github.com/Hemanth-Raja/tool-qarantine

Try a sample
Scans → json-formatter → Quarantine & run

Team Spartans
Hemanth Raja
Poojitha Katta